

optimistic - writeup

Fernando González

In Tinselwick's first magical mishap, Lottie Thimblewhisk discovers a strange peppermint-coded message whose enchanted structure hides something far more important. This challenge explores a festive ciphering mechanism and a seasonally wrapped secret. Your first step in uncovering what happened to the wandering Starshard.

part 1: understanding

we are given a python source code and an output.txt:

```
PEPPERMINT_KEYWORD = 'AR4NDOMK3Y'  
PEPPERMINT_CIPHERTEXT = [.....]  
WRAPPED_STARSHARD = '57c77ef9d669a1c7d53b5d5f867aa4c1d4ffda2e8edbc4d32153ae38bd77be44d3e70338e5c12ddd4a'
```

the python file has multiple parts. i slightly changed the variable names because they were trippy as hell lmao. in synthesis:

1. the secret is set to only have uppercase letters and numbers (which is our alphabet)
2. `weave_peppermint_square()` partitions the alphabet in a 6*6 matrix, in order, but the keyword goes first. this reminds me of a vigenère square or a transposition matrix
3. from here we make a map that associates letter -> numeric position
4. based on this square we have an encryption algorithm: for each index, the ciphertext is the numeric positions of the key and plaintext letters, added together
5. we take a random key from the alphabet and use it to encrypt the secret
6. additionally, the secret's hash is used as an AES key to encrypt what seems to be the flag

```
STARSTREAM_KEY = ''.join(random.sample(ALPHABET, k=L))  
PEPPERMINT_CIPHERTEXT = swirl_encrypt(STARSTREAM_KEY, SECRET)
```

something new i learned: `random.sample()` is a function which takes a random sample of **k unique** elements.

part 2: observations

maybe one of the first things i did was to see the peppermint square and coords map. i ran it with the keyword that we are given:

```
COORDS = {'A': '11', 'M': '21', 'E': '31', 'L': '41', 'U': '51', '2': '61', 'R': '12', 'K': '22', 'F':
```

this means every letter is mapped to a number between 11 and 66, so ciphertext numbers could be from 22 to 132

i also calculated that there are upto 1296 key-plaintext combinations, and that each sum can be made by up to 12 different of them. i wanted to try out bruteforce (especially cuz the key was of length 36), but 12^{36} is a giant number, and to verify i'd need to run AES every time

i also realized that the key size is the same as the alphabet, so that random sample is actually a permutation

part 3: getting closer

one of the things i've learned in competitive programming is that some processes are useful to see them in reverse. we see that

$$c_i = k_i + p_i$$

so, for example, we can ask ourselves: which combinations yield 22? turns out only 11+11 works. so if we find a $c_i = 22$, we can be sure $k_i = 11$ for that i .

```
for i in range(len(PEPPERMINT_CIPHERTEXT)):
    val = PEPPERMINT_CIPHERTEXT[i]
    if val != 22: continue
    print(val, i%L, i)
```

i worked through the indices and found:

```
22 22 1390
22 22 1678
22 22 1750
22 22 2002
22 22 2146
22 22 2362
22 22 2614
22 22 3010
```

so $k_{22} = 11$. now looking at value 23, we can have 11 or 12 as possible keys. we can see:

```
23 22 526
23 22 634
23 22 778
23 22 850
23 22 886
23 19 919
23 19 1279
23 22 1318
23 19 1567
23 19 1747
23 19 2143
23 19 2863
23 19 2971
```

there are 2 indices, but one of them is already taken. so k_{22} is still 11, thus $k_{19} = 12$.

if we go in an increasing order, this pattern repeats itself, so we'll keep finding unique correspondences. so i built a code to do this programatically:

```
import string, re, random, hashlib
from Crypto.Util.Padding import pad
from Crypto.Cipher import AES
```

```
WRAPPED_STARSHARD = '57c77ef9d669a1c7d53b5d5f867aa4c1d4ffda2e8edbc4d32153ae38bd77be44d3e70338e5c12ddd4a'
PEPPERMINT_CIPHERTEXT = [...]
```

```
# PEPPERMINT_CIPHERTEXT.sort()
# PEPPERMINT_CIPHERTEXT.reverse()
```

```
print(set(PEPPERMINT_CIPHERTEXT))
print(len(PEPPERMINT_CIPHERTEXT))
```

```
L = 36
a = set(PEPPERMINT_CIPHERTEXT)
```

```
correspondence = [None for i in range(L)]
```

```

COORDS = {'A': '11', 'M': '21', 'E': '31', 'L': '41', 'U': '51', '2': '61', 'R': '12', 'K': '22', 'F':
COORDS_VALS = [int(x) for x in list(COORDS.values())]
COORDS_INVERSE = { int(v): k for k,v in COORDS.items() }
COORDS_VALS.sort()

```

```

print(COORDS_INVERSE)

```

```

for i in range(len(PEPPERMINT_CIPHERTEXT)):
    val = PEPPERMINT_CIPHERTEXT[i]
    if val != 23: continue
    print(val, i%L, i)

```

```

j = 0
for c in set(a):
    for i in range(len(PEPPERMINT_CIPHERTEXT)):
        val = PEPPERMINT_CIPHERTEXT[i]
        if val != c: continue
        if correspondence[i%L] != None: continue

        correspondence[i%L] = COORDS_VALS[j]
        j += 1

```

```

print(correspondence)

```

with this correspondence array, we can now decrypt the whole secret

```

SECRET = [c - correspondence[i%L] for i,c in enumerate(PEPPERMINT_CIPHERTEXT)]
print(SECRET[:100])

```

```

SECRET = ''.join([COORDS_INVERSE[c] for c in SECRET])
print(SECRET)

```

this gave me a very long text related to the CTF thematic, so i knew it was correct. i went ahead and decoded the flag:

```

COCOA_AES_KEY = hashlib.sha256(SECRET.encode()).digest()
STARSHARD_SCROLL = AES.new(COCOA_AES_KEY, AES.MODE_ECB).decrypt(bytes.fromhex(WRAPPED_STARSHARD))

print(STARSHARD_SCROLL)

```

```

# b'HTB{th3_s0_c4ll3d_c1ph3r_0f_n1h1l1sts}\n\n\n\n\n\n\n\n\n\n\n'

```

yay!

ps: i also conjectured the secret could be decrypted through some sort of frequency analysis similar to vigenère's cracking. i'll wait for solutions to see if that was ever done